

教學文件

程式碼註解裡有完整的公式推導，這裡放的是**程式碼講不了的東西**：流程圖、全景視角、以及做簡報時的敘事順序。

建議順序

	文件	對應程式碼	一句話
0	符號說明	<code>tools/demo_operators.py</code>	先看這份。⊙ 和 @ 都叫「乘」，但完全是兩回事
1	全景圖	—	一句話進去，到權重被改動，中間發生了什麼
2	詞表大小是怎麼決定的	<code>tokenizer/bpe.py</code>	「6400 個詞哪來的」
3	Embedding 與 scatter-add	<code>scratch/embedding.py</code>	查表為什麼是矩陣乘法，反向為什麼是相加
4	反向傳播全鏈	全部	從 <code>p - y</code> 一路走回 <code>E</code>
5	權重是怎麼被改的	<code>scratch/adamw.py</code>	反向算完之後才輪到 optimizer
6	三個最常見的誤解	—	做簡報時最值得講的三件事
7	權重與 tokenizer 綁定	<code>tests/test_tokenizer_binding.py</code>	為什麼 continual pre-training 必須用同一份詞表

單層深入

01-07 是主線敘事。下面三份把單一層拆開講到底：完整的公式推導、結構圖、以及從追蹤檔抓出來的真實數值。做簡報時每一份都可以獨立成一個段落。

	文件	對應程式碼	重點
8	RMSNorm	<code>scratch/rmsnorm.py</code>	完整前向與反向推導（含商法則），以及它為什麼會放大梯度
9	Attention 的結構	<code>scratch/attention.py</code>	六個步驟逐一拆解，含因果遮罩與 softmax 完整 Jacobian
10	FFN 的結構（SwiGLU）	<code>scratch/swiglu.py</code>	閘門機制、為什麼升維再降維、參數量為何佔三分之二

附：怎麼產生講解用的素材

```
# ○ 與 @ 的差別，用最小的矩陣算給你看（含有限差分驗證）
python tools/demo_operators.py

# 逐層梯度驗證表（可直接貼進簡報）
python tests/test_layers.py          # -> traces/gradcheck.txt

# BPE 合併過程，看詞表怎麼從 3653 長到 6400
python tests/test_bpe.py             # -> traces/bpe_training.txt

# 第 1 步的完整數值軌跡（前向 + 反向 + AdamW 每個中間量）
python train.py --trace-steps 1      # -> traces/train_step_0001.txt

# 一次推論的完整前向
python generate.py --prompt "貓是一種" --trace # -> traces/inference.txt

# 證明權重與 tokenizer 綁死（用錯會比沒訓練還糟）
python tests/test_tokenizer_binding.py
```

`traces/sample/` 裡有一份預先跑好的，不用執行就能看。

轉成 PDF

```
python tools/build_pdf.py           # 合併成一份（含封面、目錄）
python tools/build_pdf.py --split   # 每份文件各一個
```

流程是 markdown → HTML（KaTeX + Mermaid）→ headless Chrome 列印。公式用 KaTeX 渲染、流程圖用 Mermaid 畫，兩者在 PDF 裡都是向量，放大不會糊。

改完文件記得跑 `python tools/check_latex.py` 驗證公式沒被工具鏈破壞。